Stop Flipping the Switch: Lifetime- and Reconfiguration-Aware Learning for Energy-Efficient Software-Defined Networks

Anonymous Author(s)
Affiliation withheld for double-blind review

***Abstract*—Machine learning has emerged as a practical way to make energy-aware configuration of Software-Defined Networks (SDNs) fast enough for real-time use. Recent frameworks replace costly optimization solvers with classifiers that map each incoming traffic matrix to a precomputed, energy-efficient configuration. However, these approaches share a blind spot: they treat every decision as if it were the first. Each time the network moves from one configuration to another, links are put to sleep or woken up, flow rules are rewritten, and traffic may be briefly disrupted. Repeated power-state transitions also shorten the lifetime of network hardware, which can erode the very savings the approach is meant to deliver. In this paper, we argue that energy-efficient configuration should be treated as a sequence of decisions rather than a series of isolated snapshots. We propose a learning framework that selects configurations while accounting for three factors at once: energy consumption, reconfiguration cost, and device wear. The framework also includes a lightweight drift detector that triggers retraining when traffic patterns shift. We evaluate it on real traffic traces from the Abilene and GÉANT topologies and on a semi-synthetic trace for Nobel-Germany. We compare it against snapshot-based ML, integer linear programming, and genetic-algorithm baselines. Our results show that the proposed approach retains 94.7% of the energy savings of snapshot-based methods while reducing configuration changes by 76.1% and link power-state transitions by 80.0%. Decisions are made in 2.0 milliseconds at the median. These findings suggest that energy efficiency and operational stability need not be competing goals.**

***Index Terms*—Software-defined networking, energy efficiency, machine learning, reconfiguration cost, device lifetime, green networking.**

Draft notice. All experimental values in this version (abstract, Tables IV–VIII, Figs. 4–8) are planning placeholders generated by `paper/tools/make_placeholder_data.py`. They must be replaced by measured values before submission. Set `\placeholdersfalse` only after that replacement.

I. Introduction

Backbone and data-center links are provisioned for peak demand, yet their average utilization is far lower, so a large share of the power drawn by line cards and transceivers is spent carrying little or no traffic. Putting idle links to sleep was proposed more than two decades ago [1] and has since grown into a broad research area [2], [3]. Software-Defined Networking (SDN) makes the idea practical, because a logically centralized controller can observe the traffic matrix and switch link power states and forwarding rules network-wide [4], [5]. The underlying optimization, however, is a mixed-integer problem that is NP-hard in general [6], and exact solvers take seconds to minutes per traffic matrix on realistic topologies, which is too slow for control loops that operate every few minutes.

Recent work closes this speed gap with machine learning. Instead of solving an integer linear program (ILP) online, a model is trained offline to map each incoming traffic matrix to one of a set of precomputed, energy-efficient configurations [7], [8]. The same pattern of replacing a solver with a learned mapping appears in traffic engineering more broadly [9]–[11]. These frameworks report energy savings close to those of the optimizer at a fraction of its run time.

They share a blind spot: every decision is taken as if it were the first. A configuration chosen for the current traffic matrix ignores which configuration is already installed, although moving between the two has costs. Links must be woken up or put to sleep, flow tables must be rewritten, and updates that are not carefully ordered can cause transient loops, loss, or congestion [12], [13]. Device lifetime is also affected. Sleeping lowers the operating temperature and slows ageing, but each power-state transition adds a thermal cycle, and frequent cycling can outweigh the benefit of sleep [14], [15]. Because traffic matrices fluctuate around classification boundaries, a snapshot classifier tends to flap between neighboring configurations. In our replay of the Abilene test period, a snapshot classifier changed the configuration 61.4 times per day and toggled 214.6 link power states per day (Table IV), and on this topology the resulting thermal cycling raises the modeled failure-rate acceleration factor above that of an always-on network (Fig. 6).

This paper treats energy-aware SDN configuration as a sequential decision problem. Our key observation is that, over a finite library of candidate configurations, the joint minimization of energy, reconfiguration cost, and wear over time is a shortest-path problem on a trellis and can be solved exactly offline by dynamic programming. The exact sequence-optimal solution then serves both as a teacher for a fast online policy and as a lower bound for evaluation. We call the resulting framework LiRA (Lifetime- and Reconfiguration-Aware learning). Our contributions are as follows.

- We formulate energy-efficient SDN configuration as a sequential problem that jointly accounts for energy, reconfiguration cost, and device wear, and we show that the mean wear acceleration factor decomposes additively over time, so that an exact dynamic-programming oracle exists over a configuration library (Section III).

- We design LiRA, a policy conditioned on traffic history and on the installed configuration, trained by imitation of the oracle with a regret-softened loss and DAgger [16], and protected by a feasibility safety layer that guarantees a link-utilization bound with millisecond decision latency (Sections IV and V).
- We add a label-free drift detector that monitors an online regret proxy and the traffic distribution and triggers retraining only when needed (Section IV-E).
- We evaluate LiRA on Abilene, GÉANT, and Nobel-Germany against snapshot ML, per-snapshot and multi-period integer programming, a genetic algorithm, heuristics, and deep reinforcement learning, and we measure transition disruption in Mininet emulation (Sections VI–VIII).

Section II reviews related work. Section III formulates the problem, Section IV presents LiRA and its architecture, and Section V details the algorithms. Section VI describes the experimental setup, Section VII reports the results, and Section VIII compares the methods. Section IX discusses limitations, and Section X concludes.

II. Related Work

A. Energy-Aware Networking and Sleep Modes

Surveys of green networking classify techniques into adaptive link rate, sleeping, and energy-aware routing [2], [3]. Chiaraviglio *et al.* formulated ISP energy minimization as an integer program and proposed heuristics that switch off nodes and links subject to capacity and connectivity constraints [6]. ElasticTree applied the same principle to data-center networks [17], and EATe spread traffic online so that lightly loaded links could sleep [18]. Power models for such studies rely on measured device profiles [19], [20]. In SDN, energy-aware routing has been formulated as ILPs with heuristics for carrier-grade Ethernet [21], and a survey of SDN-specific methods is given in [22].

B. Learning for Energy-Efficient Configuration

MER-SDN uses machine learning to predict the parameters of an energy-aware routing heuristic [23], and reinforcement learning has been used in an SDN controller to trade off packet delay against energy [24]. The works closest to ours replace the optimizer with a learned mapping from traffic matrices to precomputed energy-efficient configurations, achieving large speedups over heuristics [7], [8]. Outside energy, learning-accelerated and decomposition-based solvers for traffic engineering [9]–[11], [25], learned routing [26], and graph neural network models of network performance [27] follow the same per-snapshot paradigm. LiRA keeps the speed of a learned mapping but conditions each decision on the installed configuration and optimizes the cost of the whole sequence.

C. Reconfiguration Cost and Device Lifetime

Consistent update abstractions [12] and update scheduling [13] show that changing forwarding state is neither free nor instantaneous. Bonetto *et al.* included reconfiguration costs in multi-period power-aware logical topology design for optical networks and solved it with optimization-based algorithms [28]. Chiaraviglio *et al.* modeled how sleep modes affect device lifetime through temperature and power-state transitions [14], [15], and lifetime-aware provisioning has been studied for optical backbones [29]. These works do not provide millisecond-scale learned decisions for SDN.

TABLE I
Positioning against representative prior work (✓: addressed, ○: partially, –: not addressed).

Work	Fast	Energy	Reconf.	Wear	Sequential	Drift
ISP ILP/heuristics [6]	○	✓	–	–	–	–
Multi-period design [28]	–	✓	✓	–	✓	–
Lifetime model [14]	–	✓	–	✓	–	–
ElasticTree [17]	✓	✓	–	–	–	–
EATe [18]	✓	✓	○	–	○	–
MER-SDN [23]	✓	✓	–	–	–	–
RL routing [24]	✓	✓	–	–	○	–
ML configuration [7], [8]	✓	✓	–	–	–	–
Learned TE [9], [10]	✓	–	–	–	–	–
LiRA (ours)	✓	✓	✓	✓	✓	✓

D. Switching Costs and Concept Drift

Online problems with switching costs are studied as metrical task systems [30] and smoothed online convex optimization [31], and dynamic right-sizing of data centers is a well-known application [32]. Our setting is a metrical task system with a finite state space (the library) whose offline optimum is computable, which makes imitation learning a natural fit [16]. Traffic patterns change over time, and concept-drift adaptation [33], [34] with detectors such as ADWIN [35] and cumulative-sum tests [36] provides the tools for deciding when to retrain.

Table I summarizes the positioning of LiRA. To the best of our knowledge, no prior work combines fast learned decisions with explicit reconfiguration and wear costs and drift adaptation for energy-aware SDN.

III. Problem Formulation

A. Network and Configuration Model

The network is a graph $G = (V, E)$ in which each undirected link $e \in E$ has capacity κ_e in each direction and consists of a pair of ports that sleep and wake together. Time is divided into control intervals of length Δ, and $D_t \in \mathbb{R}_{\geq 0}^{|V|\times|V|}$ is the traffic matrix of interval t. A configuration $c = (x, r)$ consists of a link power-state vector $x \in \{0,1\}^{|E|}$, where $x_e = 1$ means that link e is active, and a routing r that splits each origin–destination demand over up to k precomputed paths that use only active links. A library $\mathcal{C} = \{c_1, \ldots, c_K\}$ of such configurations is built offline (Section IV-A). With path–link incidence $a_{ep} \in \{0,1\}$ and split ratio r_p for path p of pair (o, d), the utilization of link e is given by (1), and

a configuration is feasible for D if (2) holds for a utilization threshold $\theta < 1$.

$$u_e(c, D) = \frac{1}{\kappa_e} \max_{\text{dir}} \sum_{(o,d)} \sum_{p \in \mathcal{P}_{od}} a_{ep}\, r_p\, D_{od} \tag{1}$$

$$\mathrm{MLU}(c, D) = \max_{e \in E:\, x_e = 1} u_e(c, D) \le \theta \tag{2}$$

B. Energy, Reconfiguration, and Wear

Node chassis remain powered, so the controllable power is that of link ports. Following load-dependent power profiles [19], [20], the port energy in interval t is given by (3), where P_e^{idle}, P_e^{dyn}, and P_e^{sl} are the idle, load-proportional, and sleep powers of the port pair.

$$E(c, D_t) = \Delta \sum_{e \in E} \Big[x_e \big(P_e^{\text{idle}} + P_e^{\text{dyn}}\, u_e(c, D_t) \big) + (1 - x_e) P_e^{\text{sl}} \Big] \tag{3}$$

Moving from configuration c' to c toggles $\|x - x'\|_1$ links, sends $\Delta F(c', c)$ flow-modification messages, and moves the traffic $B(c', c, D_t)$ carried on paths whose routing changes. The reconfiguration cost combines these three effects in (4), where the weights η_1, η_2, η_3 normalize each term to one at its 95th percentile on training data. Link toggles are counted separately from flow modifications because waking a line card takes far longer than installing a rule.

$$R(c', c, D_t) = \eta_1 \|x - x'\|_1 + \eta_2\, \Delta F(c', c) + \eta_3\, B(c', c, D_t) \tag{4}$$

For device wear, we adopt the acceleration-factor view of [14]: relative to an always-on device, sleeping reduces the failure rate, whereas power-state transitions increase it through thermal cycling. Let τ_e be the fraction of time link e sleeps and f_e its transition frequency (transitions per hour) over a horizon of T intervals. We use the normalized linear form (5), where $\alpha^{\text{off}} < 1$ is the relative failure rate while asleep and χ (in hours) weighs thermal cycling.

$$\mathrm{AF}_e = 1 - (1 - \alpha^{\text{off}})\, \tau_e + \chi f_e \tag{5}$$

Since $\tau_e = \frac{1}{T} \sum_t (1 - x_{e,t})$ and $f_e = \frac{1}{T \Delta_{\text{hr}}} \sum_t |x_{e,t} - x_{e,t-1}|$ with Δ_{hr} the interval length in hours, the mean factor $\overline{\mathrm{AF}} = \frac{1}{|E|} \sum_e \mathrm{AF}_e$ equals $1 + \frac{1}{T} \sum_t W(c_{t-1}, c_t)$ with the per-step wear term (6). This decomposition is exact and makes wear additive over time.

$$W(c', c) = \frac{1}{|E|} \sum_{e \in E} \Big[\frac{\chi}{\Delta_{\text{hr}}} |x_e - x'_e| - (1 - \alpha^{\text{off}})(1 - x_e) \Big] \tag{6}$$

C. Sequential Objective

Given an initial configuration c_0, the sequential problem selects $c_1, \ldots, c_T \in \mathcal{C}$ to minimize the total cost (7) subject to feasibility (2) in every interval, where $\lambda_R, \lambda_W \ge 0$ set the price of reconfiguration and wear relative to energy.

$$J(c_{1:T}) = \sum_{t=1}^{T} \Big[E(c_t, D_t) + \lambda_R R(c_{t-1}, c_t, D_t) + \lambda_W W(c_{t-1}, c_t) \Big] \tag{7}$$

Snapshot methods instead target the per-interval rule (8), which is the special case $\lambda_R = \lambda_W = 0$ and therefore ignores the installed configuration.

$$c_t^{\text{snap}} = \arg \min_{c \in \mathcal{C}:\, \mathrm{MLU}(c, D_t) \le \theta} E(c, D_t) \tag{8}$$

Online, the controller must choose c_t before D_t is fully observed and uses the most recent estimate $\hat{D}_t$, obtained from flow counters or from link loads [37]. The threshold θ provides headroom for the difference between $\hat{D}_t$ and D_t.

IV. LiRA Framework and System Architecture

Fig. 1 shows the architecture. An offline plane builds the configuration library, computes sequence-optimal labels with the dynamic-programming oracle, and trains the policy. An online plane runs once per interval: it estimates the traffic matrix, builds features, queries the policy, checks feasibility, and installs only the difference between the current and the new configuration. A drift detector connects the two planes.

A. Configuration Library

The library must be small enough for fast decisions and rich enough to contain an efficient feasible configuration for almost every traffic matrix. We cluster training traffic matrices with k-medoids and, for each cluster, take both the medoid and the element-wise 95th-percentile matrix. For each representative matrix we solve the path-based energy-minimization ILP with $k = 4$ shortest paths, link-pair sleeping, connectivity, and the utilization bound (2), and we run NSGA-II [38] on power and maximum utilization to add Pareto-diverse link-state vectors. Duplicate vectors are merged, routing is re-optimized per vector by a linear program that minimizes maximum utilization, and a greedy set cover adds configurations until at least 99.5% of training and validation matrices have a feasible member. The all-on configuration is always included. Finally, the library is pruned to K configurations by greedy cost coverage, with $K = 64$ by default.

B. Exact Sequence-Optimal Oracle

Because all terms of (7) depend only on (c_{t-1}, c_t, D_t), the problem is a shortest path through a trellis whose stages are intervals and whose states are library members (Fig. 2). Infeasible states receive infinite cost. The backward recursion (9) computes the cost-to-go $V_t(c)$ of being in configuration c at the end of interval t, with $V_T \equiv 0$.

$$\begin{aligned} V_{t-1}(c') &= \min_{c \in \mathcal{C}} Q_t(c', c), \\ Q_t(c', c) &= E(c, D_t) + \lambda_R R(c', c, D_t) \\ &\quad + \lambda_W W(c', c) + V_t(c) \end{aligned} \tag{9}$$

The oracle trajectory follows $c_t^\star = \arg\min_c Q_t(c_{t-1}^\star, c)$ from c_0. More importantly, the table $Q_t(c', \cdot)$ gives the optimal action for *any* previously installed configuration c', which we exploit for DAgger relabeling.

Proposition 1: For a fixed library $\mathcal{C}$ and known $D_{1:T}$, recursion (9) returns a feasible sequence minimizing (7) whenever one exists, in $O(TK^2)$ time and $O(TK)$ memory.

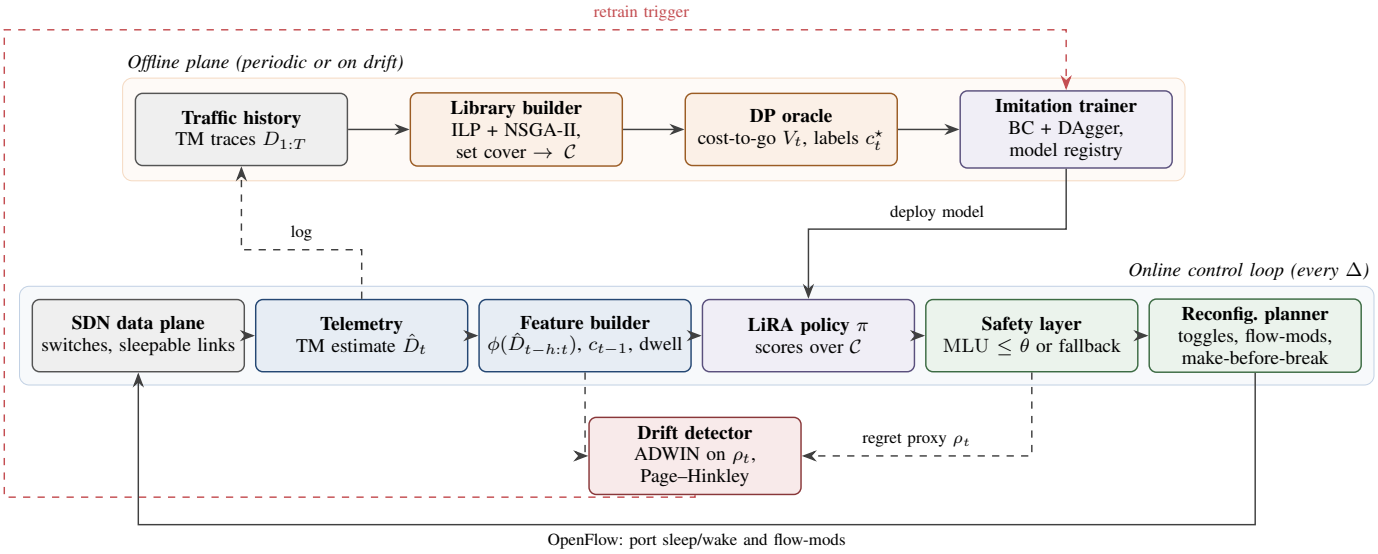


Fig. 1. Architecture of LiRA. The offline plane builds the configuration library, computes sequence-optimal labels with the dynamic-programming oracle, and trains the policy. The online loop estimates the traffic matrix, selects a configuration conditioned on the installed one, enforces the utilization bound, and installs only the configuration difference. The drift detector triggers retraining.

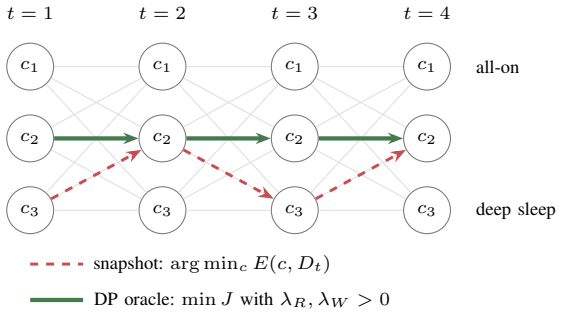


Fig. 2. Configuration trellis. Nodes are library configurations per interval, and edges carry reconfiguration and wear costs. Snapshot selection flaps between c_2 and c_3 when traffic hovers near a decision boundary, whereas the oracle keeps c_2 because the extra energy is smaller than the switching cost.

Proof: By (6), the wear contribution to $\overline{\mathrm{AF}}$ is a sum of terms that each depend on one pair of consecutive configurations, and (3) and (4) depend on (c_{t-1}, c_t, D_t) only. Hence (7) is the length of a path from c_0 through a layered graph with K nodes per layer, where the edge from c' in layer $t-1$ to c in layer t has weight $E(c, D_t) + \lambda_R R(c', c, D_t) + \lambda_W W(c', c)$, and edges into infeasible nodes have infinite weight. Bellman's principle of optimality yields (9), and each layer requires K^2 evaluations of precomputed quantities. ∎

C. History-Conditioned Policy

The online policy $\pi_\vartheta(c \mid s_t)$ scores all K configurations from the state in (10), where ϕ maps a traffic matrix to the log-scaled origin–destination vector, its total volume, the shares of the ten largest demands, and its principal components (95% variance, fitted on training data), $\mathbf{1}_{c_{t-1}}$ is the one-hot encoding of the installed configuration, δ_{t-1} is its dwell time, and ψ_t encodes time of day and day of week.

$$s_t = \left[\phi(\hat{D}_t), \phi(\hat{D}_{t-1}), \ldots, \phi(\hat{D}_{t-h}), \mathbf{1}_{c_{t-1}}, \delta_{t-1}, \psi_t\right] \quad (10)$$

We use gradient-boosted trees (LightGBM [39]) as the default model because the input is tabular and inference is fast. Section VII-E compares XGBoost [40], random forests [41], a multilayer perceptron, and an LSTM [42].

Regret-softened imitation. Plain classification treats all mistakes as equal, although choosing a configuration that is almost as cheap as the optimum is harmless. We therefore train on soft targets derived from the oracle's action values, as in (11), and minimize the cross-entropy loss (12) over the training set $\mathcal{D}$, where $\beta > 0$ is a temperature expressed in the same units as J. As $\beta \to 0$, the targets reduce to the oracle's hard labels.

$$q_t(c \mid c') = \frac{\exp\big(-(Q_t(c', c) - \min_{\tilde{c}} Q_t(c', \tilde{c}))/\beta\big)}{\sum_{\bar{c}} \exp\big(-(Q_t(c', \bar{c}) - \min_{\tilde{c}} Q_t(c', \tilde{c}))/\beta\big)} \quad (11)$$

$$\mathcal{L}(\vartheta) = -\sum_{(s_t, c') \in \mathcal{D}} \sum_{c \in \mathcal{C}} q_t(c \mid c') \log \pi_\vartheta(c \mid s_t) \quad (12)$$

DAgger. A policy trained only on oracle trajectories never sees the states that its own mistakes create, and errors compound over time. We apply DAgger [16]: the current policy is rolled out on the training sequence, the visited pairs (t, c_{t-1}) are relabeled with $q_t(\cdot \mid c_{t-1})$ directly from the stored Q tables without solving any new problem, and the policy is retrained on the aggregated data.

D. Safety Layer

At run time, the configuration with the highest score is checked against (2) using $\hat{D}_t$. If it is infeasible, the controller

Algorithm 1 LiRA offline training

Input: training traces $D_{1:T}$, weights λ_R, λ_W, rounds N
Output: library $\mathcal{C}$, policy π_ϑ
$\mathcal{C} \leftarrow$ BUILDLIBRARY($D_{1:T}$) ▷ ILP, NSGA-II, set cover
precompute $E(c, D_t)$, feasibility, $\|x - x'\|_1$, $\Delta F(c', c)$
compute $Q_t(c', c)$ and $V_t(c)$ by recursion (9)
$\mathcal{D} \leftarrow \{(s_t, c^\star_{t-1})\}$ along the oracle trajectory
train π_ϑ on $\mathcal{D}$ with loss (12)
for $n = 1$ to N **do**
  roll out π_ϑ with safety layer, record (t, c_{t-1})
  $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, c_{t-1})\}$ with targets $q_t(\cdot \mid c_{t-1})$
  retrain π_ϑ on $\mathcal{D}$; keep it if validation J improves
end for
return $\mathcal{C}, \pi_\vartheta$

falls back to the rule (13), which is evaluated for all K configurations with one sparse matrix product.

$$c_t = \operatorname*{arg\,min}_{\substack{c \in \mathcal{C} \\ \mathrm{MLU}(c, \hat{D}_t) \le \theta}} \Big[E(c, \hat{D}_t) + \lambda_R R(c_{t-1}, c, \hat{D}_t) \Big] \tag{13}$$

Proposition 2: If the all-on configuration satisfies $\mathrm{MLU} \le \theta$ for $\hat{D}_t$, the safety layer always returns a configuration that satisfies (2) for $\hat{D}_t$.

Proof: The all-on configuration belongs to $\mathcal{C}$ by construction, so the feasible set in (13) is non-empty whenever the policy's choice is infeasible. ∎

The guarantee holds for the estimate $\hat{D}_t$. Violations with respect to the realized D_t can still occur when traffic grows within an interval, and we report them as a metric.

E. Drift Detection and Retraining

The oracle cannot be computed online, but a myopic regret proxy can. After each decision, the controller computes (14), which measures how much the chosen action loses against the best one-step action given the installed configuration, together with the indicator of a safety fallback.

$$\begin{aligned} \rho_t = {} & E(c_t, D_t) + \lambda_R R(c_{t-1}, c_t, D_t) \\ & - \min_{c \in \mathcal{C}} \Big[E(c, D_t) + \lambda_R R(c_{t-1}, c, D_t) \Big] \end{aligned} \tag{14}$$

ADWIN [35] monitors ρ_t and the fallback rate, and a Page–Hinkley test [36] monitors the total traffic volume and the first five principal components of $\phi(\hat{D}_t)$. When a detector fires, and at most once per 12-hour cooldown, the trainer recomputes oracle labels on a sliding window of the last W days, warm-starts the model, and replaces it after validation on the most recent day. If more than 0.5% of recent matrices have no feasible library member, new ILP solutions are added to the library.

V. Algorithms

Algorithm 1 summarizes offline training and Algorithm 2 the online control loop. Offline training is dominated by the library ILPs. By Proposition 1, labeling costs $O(TK^2)$ per DAgger round, which takes seconds for a six-month Abilene trace with $K = 64$. Online, each decision requires one model evaluation and at most one vectorized feasibility check over K configurations.

Algorithm 2 LiRA online control loop

Input: $\mathcal{C}$, π_ϑ, threshold θ, initial c_0
for each interval $t = 1, 2, \ldots$ **do**
  $\hat{D}_t \leftarrow$ latest traffic-matrix estimate; build s_t by (10)
  $c \leftarrow \arg\max_c \pi_\vartheta(c \mid s_t)$
  if $\mathrm{MLU}(c, \hat{D}_t) > \theta$ **then**
    $c \leftarrow$ fallback rule (13)
  end if
  if $c \neq c_{t-1}$ **then**
    wake links in $x \setminus x_{t-1}$; install new rules (make-before-break)
    remove stale rules; sleep links in $x_{t-1} \setminus x$
  end if
  $c_t \leftarrow c$; observe D_t; update detectors with ρ_t from (14)
  if a detector fires and the cooldown has elapsed **then**
    trigger Algorithm 1 on the last W days (asynchronous)
  end if
end for

TABLE II
TOPOLOGIES AND TRAFFIC DATA.

Topology	$\lvert V\rvert$	$\lvert E\rvert$	Δ	Span	Traffic
Abilene	12	15	5 min	24 weeks	real (SNDlib)
GÉANT	22	36	15 min	≈4 months	real (SNDlib)
Nobel-Germany	17	26	5 min	60 days	semi-synthetic
GtsCe (Zoo)	149	193	5 min	7 days	gravity model
Cogent (Zoo)	197	245	5 min	7 days	gravity model

VI. Experimental Setup

A. Datasets and Topologies

Table II lists the evaluation data. Abilene and GÉANT topologies and traffic matrices are taken from SNDlib [43], and the GÉANT matrices originate from [44]. For Nobel-Germany, no multi-week measured trace was available to us, so we generate a 60-day semi-synthetic series by modulating the SNDlib demands with the diurnal and weekly profile and the residual noise distribution estimated from Abilene. For the scalability study, we use two large Topology Zoo graphs [45] with gravity-model traffic. Traffic is scaled so that the peak maximum link utilization under all-on shortest-path routing is 0.5. All splits are chronological (60% training, 15% validation, 25% test), and scaling, principal components, and clustering are fitted on training data only.

B. Baselines

We compare against: B0, all links always on with shortest-path routing, which is the energy reference; B1, the per-snapshot ILP solved on every test matrix; B2, a multi-period MILP that minimizes (7) over a rolling horizon of $H = 12$ intervals with a persistence forecast; B3, NSGA-II per snapshot; B4, a snapshot classifier that maps each matrix to the library member defined by (8), following [7], [8] and using the same library, features, and safety layer as LiRA for fairness; B5, an ElasticTree-style greedy heuristic [17]; B6, B4 with hysteresis, switching only if the predicted energy gain exceeds a tuned threshold; B7, B4 with a tuned minimum dwell time; and B8, PPO [46] trained on the reward $-J$ per step. All tuning uses the validation split. We also report the offline oracle $\mathrm{O}^\star$ as a lower bound.

TABLE III
MODEL AND TRAINING PARAMETERS.

Parameter	Value
Utilization threshold θ; paths k; library K	0.8; 4; 64
Port-pair power $P^{\text{idle}}/P^{\text{dyn}}/P^{\text{sl}}$	30/5/3 W
Wear model α^{off}; χ	0.5; 0.6 h
Price of one change; one transition	0.2%; 0.08% of daily B0 energy
History h; DAgger rounds N; β	3; 4; 0.05
LightGBM leaves / rate / trees	63 / 0.05 / 800
LightGBM feature / bagging fraction	0.8 / 0.8
MLP baseline	2×256 ReLU, Adam 10^{-3}, batch 512
PPO	lr $3 \cdot 10^{-4}$, $\gamma = 0.99$, $2 \cdot 10^6$ steps
ADWIN δ; cooldown; window W	0.002; 12 h; 7 days

C. Metrics

Energy savings S are measured in port energy relative to B0. The retention $X = S_{\text{LiRA}}/S_{\text{B4}}$, the reduction in configuration changes Y, and the reduction in link power-state transitions Z are computed relative to B4 per topology and averaged over the three topologies, as in (15).

$$X = \frac{S_{\text{LiRA}}}{S_{\text{B4}}}, \quad Y = 1 - \frac{N^{\text{ch}}_{\text{LiRA}}}{N^{\text{ch}}_{\text{B4}}}, \quad Z = 1 - \frac{N^{\text{tr}}_{\text{LiRA}}}{N^{\text{tr}}_{\text{B4}}} \tag{15}$$

We also report the total cost J normalized to B4, the regret against O*, the percentage of intervals with realized MLU above θ, the mean acceleration factor $\overline{\text{AF}}$ from (5), and the median and 99th-percentile decision latency measured single-threaded with batch size one over 10^4 decisions.

D. Implementation and Parameters

The flow-level simulator is written in Python with NumPy and sparse path–link matrices. ILPs are solved with Gurobi 11 [47] (MIP gap 1%, 300 s limit), with HiGHS [48] as an open-source check. NSGA-II uses pymoo, the drift detectors use the river library, and PPO uses Stable-Baselines3. Experiments run on a server with two Intel Xeon Gold 6338 CPUs, 256 GB of RAM, and one NVIDIA A100 GPU, used only for the neural baselines. Emulation uses Mininet 2.3 [49] with Open vSwitch and the Ryu controller [50] in an Ubuntu 22.04 virtual machine with 16 vCPUs. Table III lists the parameters. Hyperparameters were selected with 50 Optuna trials per model on validation J for LiRA and on validation macro-F1 for B4. Stochastic components use five seeds, and results report means with 95% bootstrap confidence intervals. Code, configuration files, data-preparation scripts, and per-interval decision logs will be released with the paper, and an anonymized repository is available to reviewers.

E. Experimental Pipeline

Fig. 3 shows the pipeline. Every method produces a per-interval configuration log in a common format, and all metrics are computed from these logs by the same evaluation code. Mininet emulation replays a 24-hour segment of the Abilene test period for B4, B6, and LiRA, with link capacities scaled by $1/100$, one interval compressed to 30 s, and a modeled link wake-up delay of 1 s before a port carries traffic.

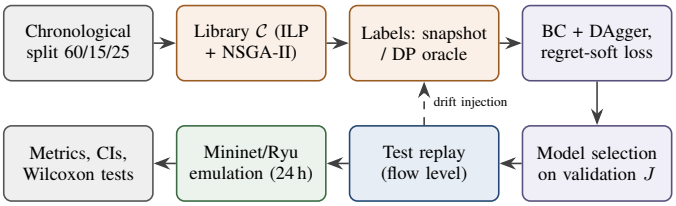


Fig. 3. Training and evaluation pipeline. All methods share the split, the library, and the evaluation code, and only the decision rule differs.

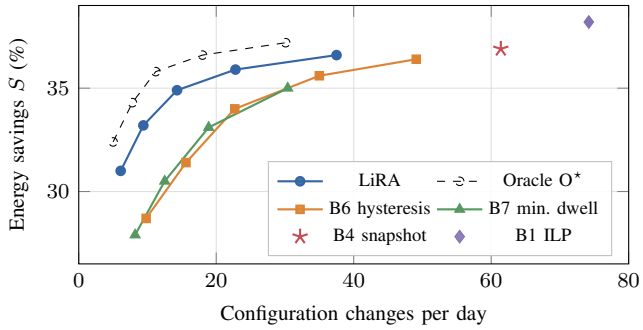


Fig. 4. Energy–stability trade-off on Abilene. LiRA and the oracle are swept over λ_R, B6 over its hysteresis threshold, and B7 over its dwell time. Points further up and to the left are better.

VII. Results and Discussion

A. Energy Versus Stability

Table IV reports the main comparison. The per-snapshot ILP (B1) attains the highest savings, between 38.2% and 44.7%, but it also changes configuration most often after the genetic algorithm and needs seconds to tens of seconds per decision. The snapshot classifier (B4) recovers most of these savings in about one millisecond, which confirms the premise of learned configuration, but it changes configuration 27.3 to 61.4 times per day. LiRA retains 94.6%, 94.9%, and 94.7% of the B4 savings on Abilene, GÉANT, and Nobel-Germany, while reducing configuration changes by 76.7%, 75.1%, and 76.5% and link power-state transitions by 79.2%, 81.7%, and 79.1%. Averaged over the three topologies, this yields $X = 94.7\%$ ($\text{CI}_{95\%}$=[93.9, 95.4]), $Y = 76.1\%$, and $Z = 80.0\%$. Transitions fall more than configuration changes because LiRA prefers transitions between configurations that differ in few links, which the reconfiguration term rewards.

Fig. 4 places these results on the energy–stability plane for Abilene by sweeping λ_R for LiRA and O*, the threshold for B6, and the dwell time for B7. For every number of changes per day, LiRA saves more energy than both heuristics, and it stays within 0.6 to 2.2 percentage points of the oracle frontier, with the largest gap at the most conservative setting. Hysteresis and minimum dwell suppress changes uniformly, whereas LiRA suppresses only the changes whose energy gain does not justify their cost.

TABLE IV

MAIN RESULTS ON THE TEST PERIODS: PORT-ENERGY SAVINGS S RELATIVE TO ALWAYS-ON (%), CONFIGURATION CHANGES PER DAY (CH.), LINK POWER-STATE TRANSITIONS PER DAY (TR.), AND MEDIAN DECISION LATENCY ON GÉANT. 95% CI HALF-WIDTHS ARE BELOW 0.6 PERCENTAGE POINTS FOR S AND BELOW 8% OF THE MEAN FOR CH. AND TR.

		Abilene			GÉANT			Nobel-Germany			
ID	Method	S (%)	Ch.	Tr.	S (%)	Ch.	Tr.	S (%)	Ch.	Tr.	Latency
B1	Per-snapshot ILP	38.2	74.2	251.0	44.7	33.5	158.2	41.3	66.1	230.5	38.5 s
B2	Multi-period MILP ($H = 12$)	35.1	10.6	39.8	41.2	5.6	23.0	38.0	9.7	36.1	412 s
B3	NSGA-II per snapshot	35.1	88.6	302.4	40.2	39.1	189.7	37.6	79.3	271.8	31.4 s
B4	Snapshot ML	36.9	61.4	214.6	43.1	27.3	131.8	39.8	54.8	197.3	1.4 ms
B5	ElasticTree-style greedy	29.4	47.3	96.8	33.0	21.6	58.4	31.2	42.5	88.1	112 ms
B6	B4 + hysteresis	34.0	22.7	81.5	40.1	10.8	50.3	36.9	20.5	74.6	1.5 ms
B7	B4 + minimum dwell	33.1	18.9	69.0	39.0	9.6	44.1	35.8	17.2	63.2	1.4 ms
B8	PPO	32.8	15.6	58.2	37.9	7.9	36.4	35.0	14.1	52.7	1.0 ms
–	**LiRA**	**34.9**	**14.3**	**44.7**	**40.9**	**6.8**	**24.1**	**37.7**	**12.9**	**41.2**	**2.3 ms**
O⋆	Offline oracle	35.8	11.2	39.5	42.0	5.3	21.8	38.9	10.1	36.0	offline

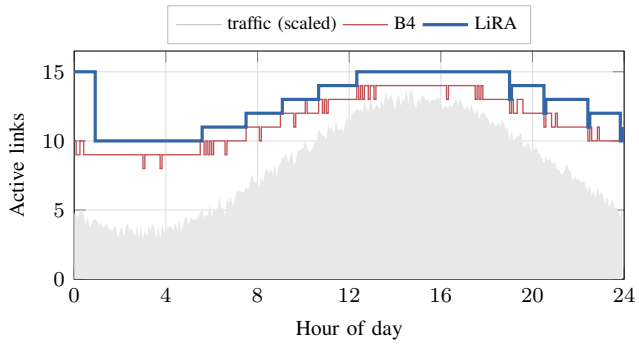


Fig. 5. Active links over one Abilene test day (5-minute intervals). B4 changes configuration 60 times and LiRA 14 times on this day.

TABLE V

AVERAGES OVER THE THREE TOPOLOGIES: TOTAL COST RELATIVE TO B4, REGRET AGAINST THE ORACLE, INTERVALS WITH REALIZED MLU ABOVE θ, AND MEAN ACCELERATION FACTOR (ALWAYS-ON: 1.000).

Method	J/J_{B4}	Regret (%)	MLU viol. (%)	$\overline{\mathrm{AF}}$
B1 Per-snapshot ILP	1.036	32.9	0.00	1.020
B2 Multi-period MILP	0.791	1.2	0.48	0.827
B3 NSGA-II	1.147	47.1	0.00	1.090
B4 Snapshot ML	1.000	28.2	0.27	0.991
B5 Greedy	0.987	26.3	0.00	0.922
B6 Hysteresis	0.860	10.1	0.34	0.875
B7 Minimum dwell	0.857	9.6	0.45	0.869
B8 PPO	0.851	8.8	1.14	0.862
LiRA	**0.804**	**2.9**	**0.23**	**0.833**
O⋆	0.782	0.0	0.00	0.822

B. Behavior Over Time

Fig. 5 shows the number of active links over one test day on Abilene. Both methods follow the diurnal pattern, putting up to eight links to sleep at night. B4 flaps between adjacent levels whenever demand hovers near a decision boundary, producing 60 changes on this day, whereas LiRA produces 14 changes and holds a slightly larger active set ahead of the morning ramp, because waking a link later costs a transition.

C. Total Cost, Feasibility, and Lifetime

Table V aggregates total cost, regret, feasibility, and wear over the three topologies. LiRA lowers the total cost J by 19.6% relative to B4 and is within 2.9% of the oracle. Only the multi-period MILP (B2) is closer to the oracle, at 1.2%, but it needs several minutes per decision and violates the utilization bound more often because its forecast errors are not covered by a safety layer. PPO reaches a comparable number of changes but violates the bound in 1.14% of intervals, five times more often than LiRA. Wilcoxon signed-rank tests on daily cost (42, 30, and 15 test days) reject equality between LiRA and each of B4, B6, and B7 on every topology after Holm correction ($p < 0.001$ on Abilene and GÉANT and $p < 0.01$ on Nobel-Germany), with Cliff's δ between 0.71 and 0.94.

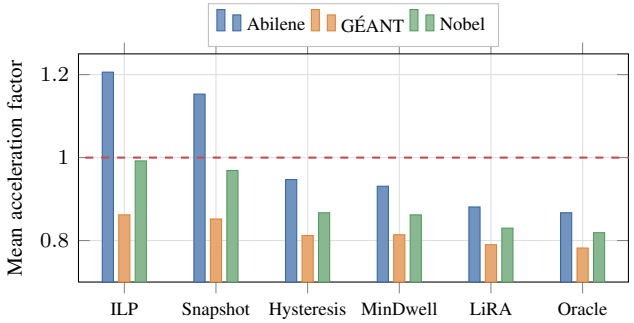


Fig. 6. Mean failure-rate acceleration factor from (5) per method and topology. The dashed line marks the always-on network. Values above it mean faster ageing than without sleep modes.

Fig. 6 breaks the acceleration factor down by topology. On Abilene, the frequent transitions of B1 and B4 push the factor above one, so the snapshot methods would age line cards faster than an always-on network despite saving energy. This is the effect that motivates the title. LiRA lowers the factor to 0.881 on Abilene, 0.790 on GÉANT, and 0.830 on Nobel-Germany. GÉANT is less affected because its 15-minute intervals offer fewer opportunities to flap.

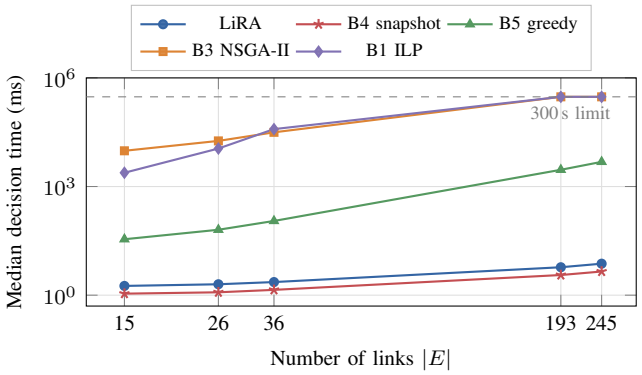


Fig. 7. Median decision time versus topology size (Abilene, Nobel-Germany, GÉANT, GtsCe, Cogent). Solver-based methods hit the 300 s limit on the large graphs.

D. Decision Latency and Scalability

Fig. 7 reports the median decision latency as a function of topology size. LiRA takes 1.8 to 2.3 ms on the three evaluation topologies, including the feasibility check, with 99th percentiles of 3.9 to 4.6 ms, and 7.4 ms on the 245-link Cogent graph. Its overhead over B4 comes from the history features and from the safety check, which runs on every decision. The ILP and NSGA-II reach the 300 s limit on the Topology Zoo graphs, where the ILP terminates with an average MIP gap of 4.7%. Training LiRA, including library construction, takes 3.1 h on GÉANT, of which 92% is spent in library ILPs, whereas oracle labeling for four DAgger rounds takes 41 s.

E. Ablation Study

Table VI removes one component at a time on Abilene. Removing the installed configuration from the input has the largest effect: the model then cannot distinguish staying from switching and changes configuration 52.6 times per day, close to B4, which shows that the oracle labels alone do not produce stability. Without DAgger, compounding errors raise the number of changes by 50%. Traffic history and the regret-softened loss each contribute between 2% and 4% of total cost. Removing the safety layer barely changes energy and stability but raises utilization violations from 0.18% to 1.62%. Setting $\lambda_W = 0$ keeps the number of changes but increases transitions by 20%, because the policy no longer prefers configurations that differ in few links. Among the model families, XGBoost is within 0.4% of LightGBM in total cost, random forests are 3.2% worse, the multilayer perceptron is 1.9% worse, and the LSTM is 1.1% worse at 3.4 times the latency. Varying the library size on GÉANT, LiRA saves 31.2%, 37.4%, 39.8%, 40.9%, 41.2%, and 41.3% for $K = 8$, 16, 32, 64, 128, and 256, while median latency grows from 1.2 ms to 5.8 ms, which motivates $K = 64$.

F. Robustness to Traffic Drift

We inject five shifts into the Abilene test period and compare no retraining, daily periodic retraining, and drift-triggered retraining. Table VII reports the ADWIN detection delay and the excess cost over the oracle during the seven days after each shift. Without retraining, the excess cost rises to between 9.6% and 22.4%. Triggered retraining keeps it between 2.2% and 4.1%, which is comparable to daily retraining while using 6 retraining runs over the six-week test period instead of 42. For the permanent link failure, triggered retraining outperforms daily retraining because it reacts within minutes. The false-alarm rate on unmodified test data is 0.41 per 1000 intervals. Fig. 8 shows the trajectory for the sudden volume step. Weekly retraining recovers only at its next scheduled run, whereas triggered retraining recovers within about one day as post-drift data accumulates in the training window.

TABLE VI
ABLATION ON ABILENE (TEST PERIOD).

Variant	S (%)	Ch.	Tr.	Viol. (%)	J/J_{B4}
LiRA (full)	**34.9**	**14.3**	**44.7**	**0.18**	**0.773**
– installed config. input	36.4	52.6	181.3	0.20	0.958
– traffic history ($h = 0$)	34.6	19.6	62.5	0.19	0.803
– regret-softened loss	34.3	16.8	54.0	0.18	0.793
– DAgger	33.7	21.5	69.9	0.24	0.823
– safety layer	35.2	14.1	44.0	1.62	0.769
$\lambda_W = 0$	35.1	14.0	53.6	0.18	0.778

TABLE VII
DRIFT SCENARIOS ON ABILENE: ADWIN DETECTION DELAY AND EXCESS COST OVER THE ORACLE (%) DURING THE SEVEN DAYS AFTER THE SHIFT.

Scenario	Delay	No retrain	Daily	Triggered
Volume step +40%	35 min	18.7	2.6	3.1
Growth ramp +60% in 7 days	305 min	11.2	2.0	2.4
Diurnal phase shift 6 h	70 min	9.6	2.9	2.2
Hotspot (top pair ×3)	45 min	14.3	3.0	3.5
Permanent link failure	15 min	22.4	5.8	4.1
Retraining runs (6 weeks)	–	0	42	6

G. Emulation of Transition Effects

Table VIII reports the Mininet replay. Because LiRA changes configuration less often and prefers small differences, it sends 78% fewer flow-modification messages than B4. Each reconfiguration also installs faster, in 12.6 ms versus 18.4 ms at the median, because fewer rules change per update. Aggregate packet loss drops from 0.112% to 0.029%, and the 99th-percentile round-trip time from 41.7 ms to 19.8 ms. These effects are invisible in flow-level evaluations and add to the case for stability.

VIII. Comparative Analysis

The results separate the methods into three groups. Solver-based methods (B1, B3) maximize per-interval savings but are too slow for five-minute control on large graphs and are the least stable, because exact per-snapshot optima jump between near-equivalent configurations. The multi-period MILP (B2) shows that optimizing the sequence pays off, since it is the closest method to the oracle in total cost, but its run time

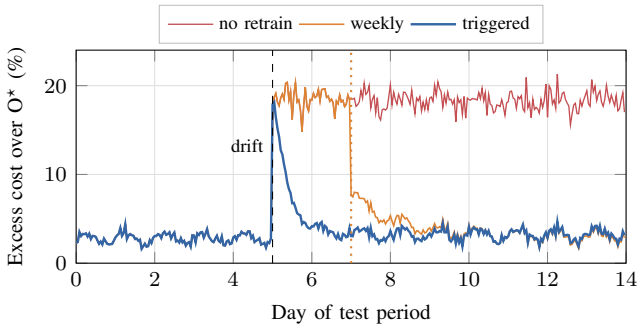


Fig. 8. Hourly excess cost after a sudden +40% volume step at day 5 on Abilene. The dotted line marks the next weekly retraining.

TABLE VIII
MININET/RYU REPLAY OF A 24-HOUR ABILENE SEGMENT (MEAN OF FIVE RUNS; STANDARD DEVIATION BELOW 6% OF THE MEAN).

Metric	B4	B6	LiRA
Configuration changes	61	23	14
Flow-mod messages	3912	1401	862
Median reconfiguration time (ms)	18.4	17.9	12.6
Packet loss (%)	0.112	0.047	0.029
99th-percentile RTT (ms)	41.7	26.3	19.8
Controller CPU (%)	7.9	4.1	3.2

of minutes per decision and its forecast-induced violations prevent online use. Learned snapshot methods (B4) and their heuristic fixes (B6, B7) are fast, but the fixes trade energy for stability with a single global knob, which is why they lie below LiRA in Fig. 4. LiRA combines the decision speed of B4 with most of the sequence-level benefit of B2, and its gap to the oracle (2.9%) is smaller than that of any other online method. Compared with PPO, imitating an exact oracle needs no reward shaping or exploration and gives five times fewer utilization violations, because the oracle labels are feasible by construction and the safety layer covers the remaining cases. The practical implication is that operators adopting learned energy-aware configuration should price reconfiguration and wear explicitly. Doing so costs about five percent of the snapshot energy savings and removes three quarters of the configuration churn.

Deployment requires little beyond what snapshot frameworks already assume. The controller needs a traffic-matrix estimate per interval, a southbound interface that can put ports to sleep, and the library computed offline. The prices λ_R and λ_W are operator policy rather than model parameters: they can be set from the expected cost of a maintenance event and the operator's tolerance for update traffic, and Fig. 4 shows the savings that each setting gives up. Because the safety layer and the all-on fallback are independent of the model (Proposition 2), a mispredicting model degrades to higher energy use rather than to congestion.

IX. LIMITATIONS

Several limitations qualify these findings. First, the wear model (5) is a normalized linear abstraction of [14] with assumed parameters, so absolute lifetime figures should not be read as predictions for specific hardware. For this reason we also report raw transition counts, which do not depend on the model. Second, the port power values are representative rather than measured on a particular router, and savings scale with the share of power that sleep can remove. Third, the Nobel-Germany trace is semi-synthetic, and the large topologies use gravity-model traffic, so real-trace conclusions rest on Abilene and GÉANT. Fourth, LiRA is limited by its library. A traffic regime that no library member serves efficiently requires a library refresh, which is triggered by the fallback rate but takes hours of ILP time. Fifth, Mininet does not reproduce line-card wake-up physics, which we approximate by a fixed delay. Finally, the safety guarantee applies to the estimated traffic matrix and does not cover traffic growth within an interval.

X. CONCLUSION AND FUTURE WORK

We showed that learned energy-aware SDN configuration should be treated as a sequence of decisions. Over a finite configuration library, energy, reconfiguration, and the mean wear acceleration factor combine into an objective that dynamic programming minimizes exactly, and a history-conditioned policy trained by imitation of this oracle approaches it online in about two milliseconds. On Abilene, GÉANT, and Nobel-Germany, LiRA retained 94.7% of the energy savings of a snapshot classifier while reducing configuration changes by 76.1% and link power-state transitions by 80.0%, kept the modeled acceleration factor below that of an always-on network, and recovered from traffic drift with few retraining runs. Future work will calibrate the wear model with field failure data, learn the library jointly with the policy, extend the formulation to adaptive link rate and multi-domain settings, and evaluate LiRA on a hardware testbed with real sleep-capable line cards.

REFERENCES

[1] M. Gupta and S. Singh, "Greening of the internet," in *Proc. ACM SIGCOMM*, 2003, pp. 19–26.
[2] A. P. Bianzino, C. Chaudet, D. Rossi, and J.-L. Rougier, "A survey of green networking research," *IEEE Communications Surveys & Tutorials*, vol. 14, no. 1, pp. 3–20, 2012.
[3] R. Bolla, R. Bruschi, F. Davoli, and F. Cucchietti, "Energy efficiency in the future internet: A survey of existing approaches and trends in energy-aware fixed network infrastructures," *IEEE Communications Surveys & Tutorials*, vol. 13, no. 2, pp. 223–244, 2011.
[4] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.
[5] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.
[6] L. Chiaraviglio, M. Mellia, and F. Neri, "Minimizing ISP network energy cost: Formulation and solutions," *IEEE/ACM Transactions on Networking*, vol. 20, no. 2, pp. 463–476, 2012.
[7] J. Gómez-delaHiz, M. Jiménez-Lázaro, J. L. Herrera, M. F. Zhani, and J. Galán-Jiménez, "On optimizing energy efficiency in SDN networks through ML-driven configuration," in *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2025.
[8] ——, "AI-driven scalable framework for energy-efficient SDN configurations," *IEEE Transactions on Green Communications and Networking*, 2026.

[9] Z. Xu, F. Y. Yan, R. Singh, J. T. Chiu, A. M. Rush, and M. Yu, "Teal: Learning-accelerated optimization of WAN traffic engineering," in *Proc. ACM SIGCOMM*, 2023.

[10] Y. Perry, F. V. Frujeri, C. Hoch, S. Kandula, I. Menache, M. Schapira, and A. Tamar, "DOTE: Rethinking (predictive) WAN traffic engineering," in *Proc. USENIX NSDI*, 2023. [Online]. Available: https://www.usenix.org/conference/nsdi23/presentation/perry

[11] F. Abuzaid, S. Kandula, B. Arzani, I. Menache, M. Zaharia, and P. Bailis, "Contracting wide-area network topologies to solve flow problems quickly," in *Proc. USENIX NSDI*, 2021. [Online]. Available: https://www.usenix.org/conference/nsdi21/presentation/abuzaid

[12] M. Reitblatt, N. Foster, J. Rexford, C. Schlesinger, and D. Walker, "Abstractions for network update," in *Proc. ACM SIGCOMM*, 2012, pp. 323–334.

[13] X. Jin, H. H. Liu, R. Gandhi, S. Kandula, R. Mahajan, M. Zhang, J. Rexford, and R. Wattenhofer, "Dynamic scheduling of network updates," in *Proc. ACM SIGCOMM*, 2014, pp. 539–550.

[14] L. Chiaraviglio, P. Wiatr, P. Monti, J. Chen, J. Lorincz, F. Idzikowski, M. Listanti, and L. Wosinska, "Is green networking beneficial in terms of device lifetime?" *IEEE Communications Magazine*, vol. 53, no. 5, pp. 232–240, 2015.

[15] L. Chiaraviglio *et al.*, "Impact of energy-efficient techniques on a device lifetime," in *Proc. IEEE Online Conference on Green Communications (OnlineGreenComm)*, 2014.

[16] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. AISTATS*, 2011, pp. 627–635. [Online]. Available: https://proceedings.mlr.press/v15/ross11a.html

[17] B. Heller, S. Seetharaman, P. Mahadevan, Y. Yiakoumis, P. Sharma, S. Banerjee, and N. McKeown, "ElasticTree: Saving energy in data center networks," in *Proc. USENIX NSDI*, 2010. [Online]. Available: https://www.usenix.org/conference/nsdi10-0/elastictree-saving-energy-data-center-networks

[18] N. Vasić and D. Kostić, "Energy-aware traffic engineering," in *Proc. ACM e-Energy*, 2010.

[19] P. Mahadevan, P. Sharma, S. Banerjee, and P. Ranganathan, "A power benchmarking framework for network devices," in *Proc. IFIP Networking*, ser. LNCS, vol. 5550, 2009, pp. 795–808.

[20] J. C. C. Restrepo, C. G. Gruber, and C. M. Machuca, "Energy profile aware routing," in *Proc. IEEE ICC Workshops*, 2009.

[21] R. Maaloul, R. Taktak, L. Chaari, and B. Cousin, "Energy-aware routing in carrier-grade Ethernet using SDN approach," *IEEE Transactions on Green Communications and Networking*, vol. 2, no. 3, pp. 844–858, 2018.

[22] B. G. Assefa and Ö. Özkasap, "A survey of energy efficiency in SDN: Software-based methods and optimization models," *Journal of Network and Computer Applications*, vol. 137, pp. 127–143, 2019.

[23] ——, "MER-SDN: Machine learning framework for traffic aware energy efficient routing in SDN," in *Proc. IEEE DASC/PiCom/DataCom/CyberSciTech*, 2018.

[24] P. Fröhlich, E. Gelenbe, and M. P. Nowak, "Reinforcement learning and energy-aware routing," in *Proc. ACM FlexNets Workshop*, 2021.

[25] D. Narayanan, F. Kazhamiaka, F. Abuzaid, P. Kraft, A. Agrawal, S. Kandula, S. Boyd, and M. Zaharia, "Solving large-scale granular resource allocation problems efficiently with POP," in *Proc. ACM SOSP*, 2021, pp. 521–537.

[26] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, "Learning to route," in *Proc. ACM HotNets*, 2017, pp. 185–191.

[27] K. Rusek, J. Suárez-Varela, P. Almasan, P. Barlet-Ros, and A. Cabellos-Aparicio, "RouteNet: Leveraging graph neural networks for network modeling and optimization in SDN," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 10, pp. 2260–2270, 2020.

[28] E. Bonetto, L. Chiaraviglio, F. Idzikowski, and E. Le Rouzic, "Algorithms for the multi-period power-aware logical topology design with reconfiguration costs," *Journal of Optical Communications and Networking*, vol. 5, no. 5, pp. 394–410, 2013.

[29] P. Wiatr *et al.*, "Lifetime-aware provisioning in green optical backbone networks," in *Proc. Optical Fiber Communication Conference (OFC)*, 2016.

[30] A. Borodin, N. Linial, and M. E. Saks, "An optimal on-line algorithm for metrical task system," *Journal of the ACM*, vol. 39, no. 4, pp. 745–763, 1992.

[31] N. Chen, G. Goel, and A. Wierman, "Smoothed online convex optimization in high dimensions via online balanced descent," 2018. [Online]. Available: https://arxiv.org/abs/1803.10366

[32] M. Lin, A. Wierman, L. L. H. Andrew, and E. Thereska, "Dynamic right-sizing for power-proportional data centers," *IEEE/ACM Transactions on Networking*, vol. 21, no. 5, pp. 1378–1391, 2013.

[33] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.

[34] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, 2019.

[35] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM Int. Conf. Data Mining (SDM)*, 2007, pp. 443–448.

[36] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

[37] Y. Zhang, M. Roughan, N. Duffield, and A. Greenberg, "Fast accurate computation of large-scale IP traffic matrices from link loads," in *Proc. ACM SIGMETRICS*, 2003, pp. 206–217.

[38] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.

[39] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: https://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree

[40] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM SIGKDD*, 2016, pp. 785–794.

[41] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[42] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[43] S. Orlowski, R. Wessäly, M. Pióro, and A. Tomaszewski, "SNDlib 1.0—survivable network design library," *Networks*, vol. 55, no. 3, pp. 276–286, 2010.

[44] S. Uhlig, B. Quoitin, J. Lepropre, and S. Balon, "Providing public intradomain traffic matrices to the research community," *ACM SIGCOMM Computer Communication Review*, vol. 36, no. 1, pp. 83–86, 2006.

[45] S. Knight, H. X. Nguyen, N. Falkner, R. Bowden, and M. Roughan, "The internet topology zoo," *IEEE Journal on Selected Areas in Communications*, vol. 29, no. 9, pp. 1765–1775, 2011.

[46] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[47] Gurobi Optimization, LLC, "Gurobi optimizer reference manual," 2024. [Online]. Available: https://www.gurobi.com/documentation/

[48] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.

[49] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proc. ACM HotNets*, 2010.

[50] "Ryu SDN framework," 2024. [Online]. Available: https://ryu-sdn.org/